Thesis Progress Update

Eren Olmez

20 October 2026

1. From scalar routing to correctness routing

We used classical feature-based predictors and lightweight convolutional neural networks (CNNs). We changed from a scalar cost-rank target to 213 correctness targets to exploit PERTINENCE's motivation: **models have complementary strengths**. A model with lower overall accuracy can outperform a stronger model on particular inputs.

Training data. Base classifiers used 45,000 labelled images; routers used a 5,000-image development pool with separate fitting/calibration subsets. Scalar targets are batch cost ranks; correctness targets are 213 correct/wrong indicators from saved predictions. Evaluation uses 10,000 test images; deployed routing never receives true labels.

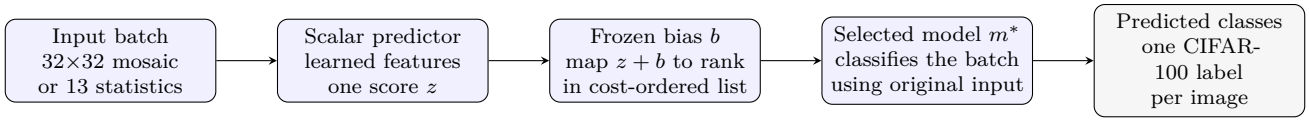
Scalar routing: training target and model selection. The classical predictors receive 13 statistics: RGB mean/std (six values), grayscale mean/std, gradient mean/std, edge density, a saturation proxy and between-image brightness variation. The scalar TinyCNN instead receives a 32×32 mosaic of 16 sampled 8×8 image tiles. Its convolutional channels are $3 \to 16 \to 24 \to 32 \to 48$, using depthwise/pointwise blocks with batch normalization/ReLU. Global average pooling gives 48 features; a linear layer produces **one scalar cost-rank score per batch**. A separate calibrated mapping converts that score into a model choice.

1. Measure candidate costs at the chosen batch size. For latency, order models from fastest to slowest; for energy, order them from least to most GPU energy. This ordering uses *cost*, not accuracy.
2. Use all candidates' saved predictions on each labelled fitting image to mark them correct or wrong.
3. Find the *cheapest candidate that was correct on that image*. If its zero-based position is r, the training target is $s = r/(M-1)$, where $M = 213$. If every candidate fails, use the reference model's position as fallback.
4. Average the per-image ranks to obtain the batch target; train the scalar CNN with mean squared error (MSE). No true labels enter inference.

For five cost-ordered candidates A–E, if A/B fail and C/D succeed, C has rank 2 and target $2/(5-1) = .50$: **a position, not 50% confidence**. Prior measurements create the list; the CNN does not generate it. One predicted score selects a position, whose model processes the original batch. More expensive does not guarantee more accurate on that input.

Scalar calibration: choosing the score-to-model mapping. Calibration is a rehearsal, not more weight training. Fix the predictor and try 1,601 biases $b \in [-1, 1]$. Each maps score z to position $\mathrm{clip}(\lfloor (z+b)M \rfloor, 0, M-1)$. Check actual accuracy using saved predictions and calibration labels, then estimate cost. An 86% reference with a 1-point allowance requires 85%. Keep the cheapest passing mapping, otherwise the reference alone. Freeze b before testing; it is a shared policy adjustment, not another CNN output.

How one number selects a model. Suppose four models have measured latencies A: 1 ms, B: 4 ms, C: 2 ms and D: 8 ms. Sorting gives the fixed list [A, C, B, D], with positions 0–3. For $z = .55$ and $b = 0$, $\lfloor .55 \times 4 \rfloor = 2$: select B. With $b = .20$, the same score gives position 3: select D. Clipping prevents an index outside the list. Thus, the score neither creates the list nor specifies an accuracy percentage. Energy uses its own measured ordering and calibration; it need not order the models the same way.

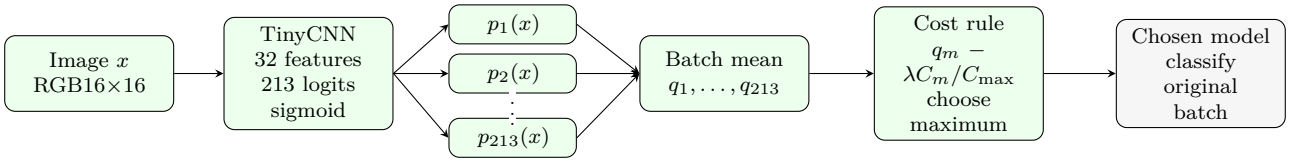


Correctness routing: targets, training and model selection.

1. *Targets:* for each labelled fitting image x and candidate m, set $y_m(x) = 1$ if the saved prediction equals the true class, otherwise 0. Each image has 213 binary targets. A wrong/B correct/C correct gives $(0, 1, 1)$: unlike scalar rank, it preserves both successful alternatives.
2. *Input and training:* the TinyCNN receives normalized RGB image content pooled to 16×16, not the true class or candidate predictions. Channels $3 \to 16 \to 24 \to 32$ use strided depthwise/pointwise ReLU blocks, without batch normalization. Global average pooling gives 32 features; a linear layer produces 213 logits, one per candidate. Sigmoid converts these to probabilities $p_m(x)$. Binary cross-entropy compares every output with its correct/wrong target. Training uses 20 epochs, Adam at 0.001 and minibatches of 64, for seeds 42/45/48.
3. *Outputs:* $p_m(x)$ estimates the probability that candidate m classifies image x correctly, not its CIFAR-100 class. Outputs need not sum to one: several candidates can succeed. Average each candidate's probabilities over the batch to obtain q_m.
4. *Selection:* let C_m be measured latency or GPU energy at this batch size, and $C_{\max}$ the largest cost. Choose the candidate maximizing $q_m - \lambda C_m / C_{\max}$. For hypothetical $q = (.70, .90, .85)$, normalized costs $(.20, 1, .40)$ and $\lambda = .10$, scores are $(.68, .80, .81)$: C wins and predicts classes for the whole batch.

Correctness calibration: choosing the cost-preference dial. Fix CNN weights and try penalties λ: smaller favours predicted success, larger favours cheaper models. Check selected models' *actual correct answers* using saved predictions and calibration labels, not CNN confidence. Keep the lowest estimated-cost setting meeting reference accuracy minus the allowance; otherwise use the reference. Calibrate latency/energy separately and freeze the policy. Passing calibration does not guarantee a test accuracy pass.

Why change the target? Scalar supervision keeps only the cheapest successful rank: it discards which other models succeeded. Correctness supervision retains those alternatives, so the cost rule can choose between candidates predicted to succeed. For example, $p_B(x) = .90$ means an estimated 90% chance that B will classify this image correctly, not a prediction of CIFAR-100 class 90. Only the selected classifier supplies the final class label. These estimates can be wrong; the measured accuracy/cost comparison, not the richer output alone, determines whether routing helps.



2. Why the 1.5-point results change sharply

The smaller model at 1.5 points is the static comparator, not necessarily the router's choice. The router selects a model per batch; validation independently selects the cheapest eligible *fixed* benchmark. Allowances are percentage-point losses (pp). Relaxing one can admit a much faster benchmark even if routing does not change.

Exact example: latency, seed 48, batch 128. R101/R50/R34 mean ResNet-101/50/34; FP16/FP32 are 16/32-bit floating point; b64 means batch 64. Both 1 and 1.5 policies use R101 FP16 for 7,424 images and R34 FP16 for 2,576. R50 FP16 is the new *benchmark*, not the router choice: its calibration loss 1.35 pp first qualifies at 1.5.

Allowance	Acc.%	Route ms/mJ per image	Original benchmark	Original L% / E%	Fixed L% / E%
1.0	84.94	1.279 / 39.78	R101 FP16	+16.00 / +15.77	+16.00 / +15.77
1.5	84.94	1.277 / 39.69	R50 FP16	-47.66 / -48.21	+16.12 / +15.97
2.0	82.25*	0.653 / 20.35	R50 FP16	+24.49 / +24.61	+57.12 / +56.92

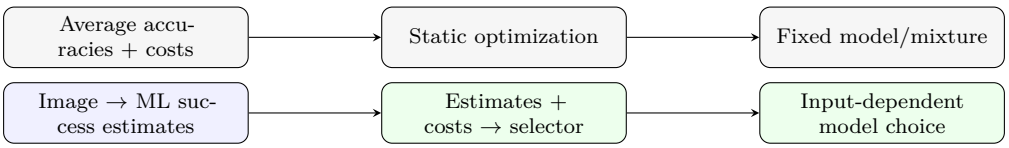
Fixed-benchmark columns: hold the 1-point reference at 1.52280 ms/image and 47.2306 mJ/image for all rows; these are renormalized savings, not new measurements. The 1/1.5-point values stay near 16%, revealing the moving denominator. At 1.5, $100(0.865 - 1.277)/0.865 = -47.66\%$: the same routing is slower than the newly eligible benchmark. At 2 points, routing becomes faster but accuracy 82.25% fails the required $85.80 - 2 = 83.80\%$ $(*)$.

Why this repeats. Across the original 120 rows, eight settings flip from positive savings at 1 point to negative at 1.5: both objectives, seed 48, batches 16/32/64/128. All eight replace R101 FP16 with R50 FP16 as benchmark. Three turn positive again at 2 points (latency b64/b128, energy b128); *all three fail accuracy.* This is not a universal middle-budget anomaly: at b8 several budgets are negative throughout.

A separate switching-cost effect. For latency/seed42/b8, the benchmark does not change, but engine loads rise 217→313 and evictions 215→311 from 1 to 1.5 points; routed time rises 69.46→99.81 s. Calibration omits these loading costs, so a relaxed allowance can select a worse switching pattern. Residual differences between identical policies remain unexplained measurement instability.

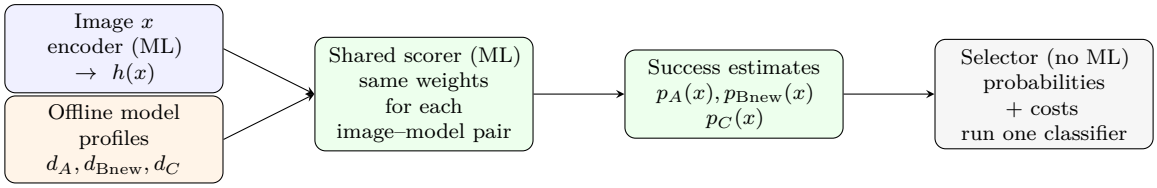
Why machine learning rather than scalar optimization?

Answer: ML supplies input-specific estimates; optimization chooses the action. Average accuracies/costs support a fixed choice; learned estimates can adapt it to the image. Our scalar router was also ML. Better choices must repay preprocessing, router and cache overhead at the required accuracy. META-DES (Cruz et al., 2015) improved accuracy on 30 small-sample problems, not GPU energy/latency in our system.



3. Can a model be replaced without retraining the whole router?

Current design versus proposal. The current 213 outputs are tied to named candidates: replacing B leaves its output predicting old B. A proposed *shared scorer* instead takes an image–model pair. For image x, an ML encoder computes visual features $h(x)$ once. Model profile d_m contains validation class-wise accuracy, precision/pruning and measured costs. The same ML scorer receives the joined vector $[h(x), d_m]$ for each available model and predicts its success probability $p_m(x)$. A non-ML cost rule then selects the classifier.



Training and changes. Offline labelled image–model pairs give correct/wrong targets for binary cross-entropy; profiles use separate evidence. No test labels enter inference. Remove 50 models by omitting their profiles/calls; the current CNN can also mask 50 outputs. Add a model by collecting validation predictions, profiling costs and inserting its profile, then validating/recalibrating the policy. No new head is needed, but accuracy is not guaranteed: test entirely held-out candidates, and fine-tune/retrain if needed. This proposal still requires initial offline training and model profiling.

Related work (untested extensions here). Tornede et al. (2020) describes instance–algorithm pair scoring; META-DES learns competence. Deep Sets (Zaheer et al., 2017) could summarize model profiles, with extra computation; changing lists does not require it.

4. Does the learned router beat uniform random selection?

Answer: yes in all 24 settings per batch size. Learned accuracy is 81.86–85.64% (b64) and 82.25–85.39% (b128), versus uniform's 36.26% and 36.25%. Learned latency is lower in all settings; energy is lower in 24/24 and 21/24. Uniform samples all 213 candidates, including accuracy failures; it is not a strong static baseline.

Comparison with 13 October. All 48 learned accuracies and selection counts match the frozen policies. Costs were remeasured: five trials per setting, 495 sweeps per batch study.

Table key. ML/R = learned/uniform; accuracy is the repeat mean. Cost pairs = median latency / GPU-energy savings (%): $100(C_{\text{benchmark}} - C_{\text{method}})/C_{\text{benchmark}}$, using the freshly measured 13 October eligible static benchmark. Positive means cheaper. L/E = objective; δ = allowance (pp); $*$ = accuracy failure versus R101 FP32 (85.80%). Every uniform row fails.

			Batch 64			Batch 128		
Obj./seed	δ	Benchmark	Acc. ML / R	ML: L / E	R: L / E	Acc. ML / R	ML: L / E	R: L / E
L/42	0.5	R101 FP16	84.93* / 36.36	+10.5 / +12.6	-505.8 / -161.6	85.23* / 36.15	+4.6 / +6.3	-215.4 / -87.6
L/42	1	R101 FP16	84.02* / 36.36	+19.5 / +22.3	-529.4 / -170.0	84.02* / 36.15	+23.0 / +26.7	-211.9 / -88.1
L/42	1.5	R101 FP16	84.02* / 36.36	+19.4 / +20.8	-555.5 / -168.4	84.02* / 36.15	+12.0 / +27.3	-286.2 / -83.5
L/42	2	R101 FP16	83.08* / 36.36	+45.0 / +45.9	-127.0 / -50.8	84.02 / 36.15	+32.2 / +33.0	-32.7 / -15.7
L/45	0.5	R101 FP16	85.52 / 36.67	+2.7 / +2.6	-151.4 / -81.2	84.93* / 37.28	+11.8 / +12.3	-28.3 / +10.6
L/45	1	R101 FP16	85.17 / 36.67	+6.7 / +6.3	-152.5 / -82.4	84.93 / 37.28	+12.0 / +12.2	-28.1 / +10.3
L/45	1.5	R101 FP16	84.58 / 36.67	+11.3 / +11.2	-152.2 / -82.8	83.93* / 37.28	+27.2 / +26.6	-28.2 / +10.1
L/45	2	R101 FP16	83.23* / 36.67	+34.8 / +34.5	-136.8 / -69.8	83.93 / 37.28	+11.1 / +11.6	-28.0 / +11.1
L/48	0.5	R101 FP16	85.41 / 35.76	+3.6 / +5.3	-142.8 / -93.4	85.19* / 35.34	+9.0 / +9.4	-17.1 / +9.6
L/48	1	R101 FP16	84.84 / 35.76	+12.6 / +14.9	-149.0 / -99.1	84.94 / 35.34	+16.2 / +16.7	-17.1 / +10.3
L/48	1.5	R50 FP16	84.84 / 35.76	-45.0 / -44.1	-312.7 / -233.1	84.94 / 35.34	-47.5 / -47.3	-106.0 / -58.9
L/48	2	R50 FP16	81.86* / 35.76	+43.4 / +43.2	-307.6 / -228.2	82.25* / 35.34	+24.7 / +27.5	-105.9 / -56.3
E/42	0.5	R101 FP16	85.23* / 36.36	+4.2 / +4.0	-126.7 / -51.2	85.39 / 36.15	+6.2 / +6.2	-32.8 / -15.6
E/42	1	R101 FP16	84.44* / 36.36	+24.0 / +24.1	-122.2 / -52.0	84.51* / 36.15	+25.1 / +25.0	-32.7 / -17.9
E/42	1.5	R101 FP16	84.44 / 36.36	+19.9 / +19.9	-386.6 / -125.4	84.51 / 36.15	+17.3 / +19.9	-212.2 / -86.1
E/42	2	R101 FP16	83.51* / 36.36	-35.4 / -8.8	-542.4 / -168.9	83.08* / 36.15	+33.0 / +36.9	-253.4 / -105.6
E/45	0.5	R101 FP16	85.27* / 36.67	+6.4 / +5.9	-149.2 / -80.2	84.51* / 37.28	+12.4 / +10.3	-16.3 / +18.6
E/45	1	R101 FP16	84.74* / 36.67	+9.4 / +9.3	-151.8 / -81.3	84.51* / 37.28	+19.2 / +19.3	-28.2 / +10.3
E/45	1.5	R101 FP16	84.34 / 36.67	+17.4 / +15.9	-146.0 / -80.0	83.93* / 37.28	+27.1 / +26.4	-28.2 / +10.0
E/45	2	R101 FP16	83.49* / 36.67	+29.0 / +29.0	-152.4 / -80.4	83.93 / 37.28	+27.1 / +26.8	-28.2 / +10.2
E/48	0.5	R101 FP16	85.64 / 35.76	+3.8 / +3.6	-139.7 / -93.2	85.39 / 35.34	+5.3 / +5.9	-17.1 / +10.0
E/48	1	R101 FP16	85.18 / 35.76	+8.3 / +7.7	-132.2 / -88.8	85.12 / 35.34	+14.3 / +14.5	-17.1 / +9.9
E/48	1.5	R50 FP16	84.66 / 35.76	-44.8 / -42.9	-303.8 / -224.5	85.12 / 35.34	-50.7 / -49.7	-105.9 / -58.1
E/48	2	R50 FP16	83.65* / 35.76	-15.7 / -13.1	-304.2 / -226.8	82.29* / 35.34	+18.7 / +21.3	-106.0 / -57.7

Uniform trials are shared across objectives/allowances; differing benchmarks explain differing savings for the same random control. GPU thermal/power limits constrain timing comparisons. No test-label tuning was used.

5. Always using a fixed 80%-pruned model

Answer: lower cost, but unacceptable accuracy loss. These completed p80 controls reach 0.93–22.55% accuracy, versus 85.80% for unpruned R101 FP32. None meets a 0.5/1/1.5/2-pp allowance.

Protocol. P80 is the nominal 80% pruning setting, not a guarantee of 80% fewer operations; INT8 is 8-bit integer precision. Seven architectures, three precisions, four batches and five repeats plus references give 440 sweeps. ShuffleNet has no pruned variant.

Comparison with 13 October. Its correctness router reached 81.86–85.64% across batches 64/128, far above p80. Against the same *type* of fixed R101 FP32 reference, earlier routing latency/energy saving ranges were: b8 −336.2 to +61.1 / −160.7 to +60.4%; b32 +60.9 to +76.3 / +61.3 to +76.5%; b64 +61.5 to +87.4 / +63.6 to +87.5%; b128 +60.0 to +81.8 / +60.5 to +82.0%. These are separate ranges across policies, including accuracy failures, not one policy achieving every endpoint. P80 saves more cost in many rows, but sacrifices most classification accuracy.

Savings baseline. Table pairs are latency saving % / GPU-energy saving % versus fresh, same-batch R101 FP32, matching the earlier *vs-FP32* definition, not the routing table's eligible-static benchmark. Accuracy is the repeat mean; costs are repeat medians. Separate sessions and observed GPU throttling limit direct cross-study timing claims.

Architecture	Precision	Acc.%	b8: L% / E%	b32: L% / E%	b64: L% / E%	b128: L% / E%
EfficientNet-B0	FP32	22.49	+84.0 / +86.2	+91.5 / +93.0	+84.6 / +84.4	+84.6 / +84.3
EfficientNet-B0	FP16	22.55	+85.4 / +88.4	+93.4 / +94.8	+90.1 / +90.0	+90.3 / +90.3
EfficientNet-B0	INT8	2.78	+78.8 / +81.3	+91.0 / +93.7	+91.2 / +90.9	+91.3 / +91.2
MobileNetV3-Large	FP32	12.96	+89.3 / +91.7	+94.8 / +96.1	+92.9 / +92.8	+92.9 / +92.9
MobileNetV3-Large	FP16	12.98	+90.0 / +92.1	+95.6 / +97.0	+95.0 / +95.0	+95.1 / +95.0
MobileNetV3-Large	INT8	5.98	+86.7 / +88.7	+94.3 / +96.1	+95.4 / +95.4	+95.6 / +95.6
MobileNetV3-Small	FP32	20.94	+89.8 / +91.6	+95.6 / +97.0	+95.5 / +95.5	+95.6 / +95.5
MobileNetV3-Small	FP16	20.97	+90.3 / +92.3	+95.6 / +97.3	+96.3 / +96.4	+96.6 / +96.5
MobileNetV3-Small	INT8	10.09	+87.5 / +90.0	+94.2 / +96.4	+96.1 / +95.8	+96.4 / +96.5
ResNet-101	FP32	1.84	+79.5 / +83.1	+84.7 / +88.1	+86.1 / +86.3	+86.6 / +86.6
ResNet-101	FP16	1.84	+87.8 / +90.2	+93.5 / +95.0	+93.2 / +93.9	+92.7 / +92.7
ResNet-101	INT8	0.93	+85.6 / +88.7	+93.3 / +95.1	+94.3 / +94.6	+94.0 / +93.9
ResNet-18	FP32	5.07	+93.7 / +94.2	+94.5 / +95.6	+95.2 / +95.5	+95.7 / +95.7
ResNet-18	FP16	5.06	+95.6 / +96.3	+97.5 / +98.1	+96.7 / +96.6	+96.9 / +96.9
ResNet-18	INT8	4.97	+94.6 / +95.2	+96.6 / +97.4	+96.3 / +96.2	+96.5 / +96.6
ResNet-34	FP32	5.47	+91.5 / +92.4	+94.0 / +95.1	+95.1 / +95.4	+95.5 / +95.5
ResNet-34	FP16	5.45	+94.1 / +95.0	+96.4 / +97.2	+96.6 / +96.8	+96.8 / +96.9
ResNet-34	INT8	4.68	+92.9 / +94.4	+96.4 / +97.4	+96.6 / +96.8	+96.9 / +96.8
ResNet-50	FP32	6.01	+89.1 / +90.8	+92.0 / +92.8	+90.4 / +90.5	+90.8 / +90.7
ResNet-50	FP16	6.02	+92.7 / +94.0	+95.8 / +96.8	+94.4 / +94.4	+94.4 / +94.3
ResNet-50	INT8	4.70	+90.9 / +92.9	+95.3 / +96.5	+95.2 / +95.1	+95.2 / +95.2

6. Per-class strengths of the unpruned FP32 models

These statistics use the entire CIFAR-100 test set: 10,000 images, balanced across 100 classes with 100 images per class. Class accuracy is the fraction of those 100 images classified correctly. Values use verified saved predictions, without new inference. All best/worst ties are shown; middle classes occupy ranks 50/51 (ties by class ID).

Architecture	Overall	Worst class	Best class(es)	Median-ranked classes
ResNet-18	79.16%	girl (45%)	motorcycle (96%)	kangaroo / lion (82%)
ResNet-34	81.82%	girl (48%)	motorcycle (100%)	rocket / telephone (84%)
ResNet-50	84.39%	girl (50%)	chimpanzee / skunk (98%)	hamster / leopard (86%)
ResNet-101	85.80%	girl (56%)	skunk (98%)	hamster / house (89%)
MobileNetV3-Small	62.38%	woman (30%)	road (93%)	fox / telephone (62%)
MobileNetV3-Large	81.34%	girl (50%)	bicycle / orange / road / tractor (96%)	house / leopard (83%)
EfficientNet-B0	82.73%	girl (53%)	road / wardrobe (97%)	table (83%); bee (84%)
ShuffleNetV2 x1.0	75.64%	otter (41%)	motorcycle (97%)	house / poppy (76%)

Complementarity example. Forest accuracy is 62% for R101, 71% for R18 and 76% for MobileNetV3-Large despite their lower overall scores. Their prior FP32 batch-32 times were 131.28, 13.64 and 6.92 ms (static boundary, not full routing). This supports PERTINENCE's motivation, not proof of routing gains; the true class is unknown at inference.[1]

The separate 71-row supplement covers all 213 variants. Frozen numerical results/source hashes remain in `numerical_snapshot.json`.

[1] Shende et al., PERTINENCE, *IEEE Access*, 2026; paper.